



DECODE LABS DATA ANALYTICS INTERNSHIP

SQL Data Analysis (QUERYING FOR TRUTH)

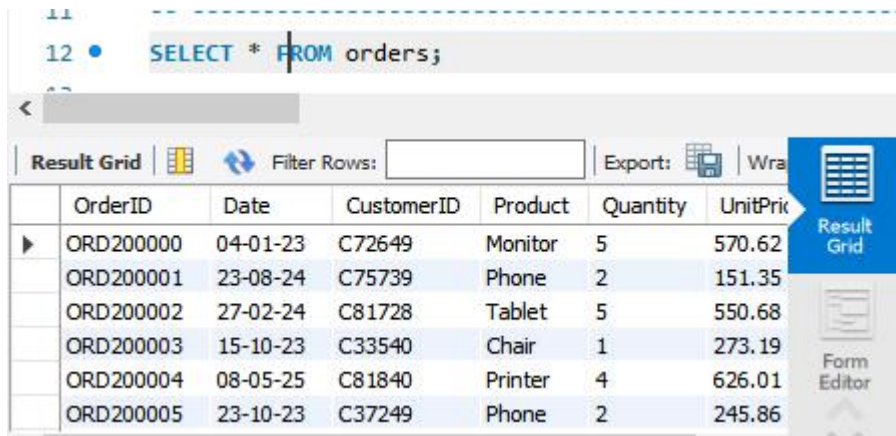
SUBMITTED BY: KHUSHI SHARMA ,B TECH COMPUTER SCIENCE AND ENGINEERING

DATASET SCALE: 1200 TRANSACTIONAL RECORDS

Task 1: Foundational View & Data Ingestion Verification

SQL Query Used: SELECT * FROM orders;

Business Insights: verified that all 1200 transactional records were successfully ingested into the database schema with zero data loss.



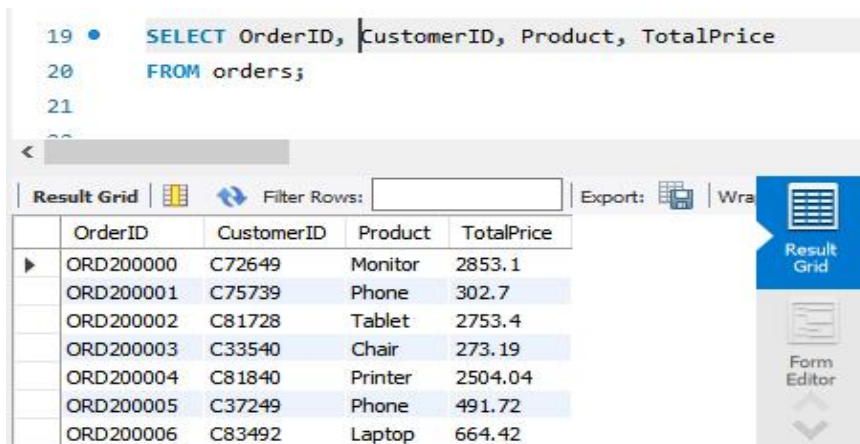
The screenshot shows a SQL query editor with the query `SELECT * FROM orders;` entered. Below the query, a 'Result Grid' is displayed, showing the first six rows of the 'orders' table. The grid has columns for OrderID, Date, CustomerID, Product, Quantity, and UnitPrice. The data is as follows:

OrderID	Date	CustomerID	Product	Quantity	UnitPrice
ORD200000	04-01-23	C72649	Monitor	5	570.62
ORD200001	23-08-24	C75739	Phone	2	151.35
ORD200002	27-02-24	C81728	Tablet	5	550.68
ORD200003	15-10-23	C33540	Chair	1	273.19
ORD200004	08-05-25	C81840	Printer	4	626.01
ORD200005	23-10-23	C37249	Phone	2	245.86

Task 2: Basic Attribute Selection (Projection Layer)

SQL Query Used: SELECT OrderID, CustomerID, Product, TotalPrice FROM orders;

Business Insight: Optimized database performance by retrieving only specific core metrics (OrderID, CustomerID, Product, TotalPrice), cutting down processing latency.



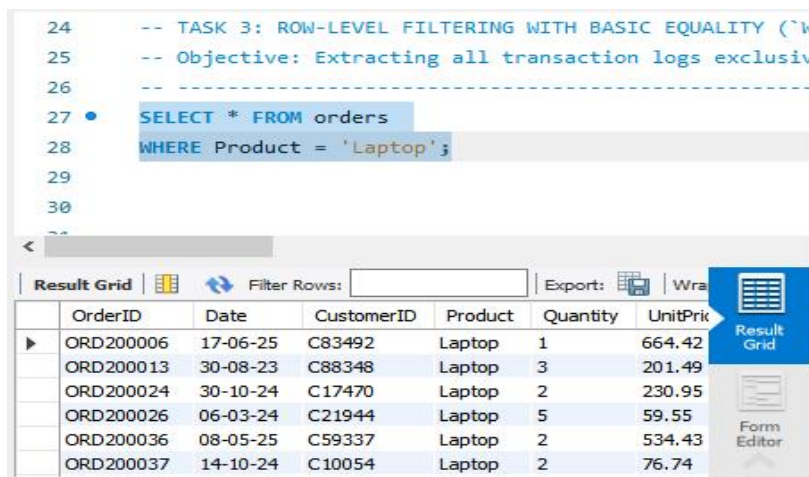
The screenshot shows a SQL query editor with the query `SELECT OrderID, CustomerID, Product, TotalPrice FROM orders;` entered. Below the query, a 'Result Grid' is displayed, showing the first six rows of the 'orders' table. The grid has columns for OrderID, CustomerID, Product, and TotalPrice. The data is as follows:

OrderID	CustomerID	Product	TotalPrice
ORD200000	C72649	Monitor	2853.1
ORD200001	C75739	Phone	302.7
ORD200002	C81728	Tablet	2753.4
ORD200003	C33540	Chair	273.19
ORD200004	C81840	Printer	2504.04
ORD200005	C37249	Phone	491.72
ORD200006	C83492	Laptop	664.42

Task 3: Row-Level Filtering (Laptop Orders)

SQL Query Used: `SELECT * FROM orders WHERE Product = 'Laptop';`

Business Insight: Isolated customer buying habits exclusively for 'Laptop' purchases to help the inventory management team.



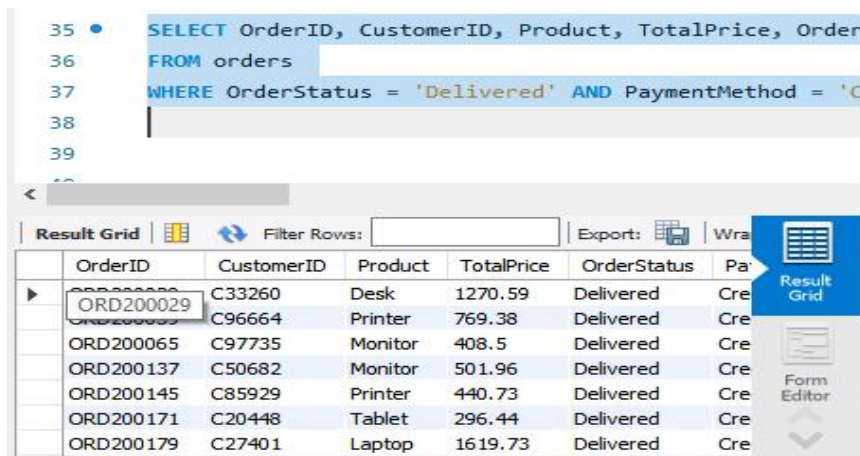
```
24  -- TASK 3: ROW-LEVEL FILTERING WITH BASIC EQUALITY (`W
25  -- Objective: Extracting all transaction logs exclusiv
26  -----
27  •  SELECT * FROM orders
28      WHERE Product = 'Laptop';
29
30
31
```

	OrderID	Date	CustomerID	Product	Quantity	UnitPrice
▶	ORD200006	17-06-25	C83492	Laptop	1	664.42
	ORD200013	30-08-23	C88348	Laptop	3	201.49
	ORD200024	30-10-24	C17470	Laptop	2	230.95
	ORD200026	06-03-24	C21944	Laptop	5	59.55
	ORD200036	08-05-25	C59337	Laptop	2	534.43
	ORD200037	14-10-24	C10054	Laptop	2	76.74

Task 4: Multi-Level Logical Filtering (Delivered via Credit Card)

SQL Query Used: `SELECT OrderID, CustomerID, Product, TotalPrice, OrderStatus, PaymentMethod FROM orders WHERE OrderStatus = 'Delivered' AND PaymentMethod = 'Credit Card';`

Business Insight: Segments high-priority, secure transactions to analyze the behavior of credit-verified premium customers.



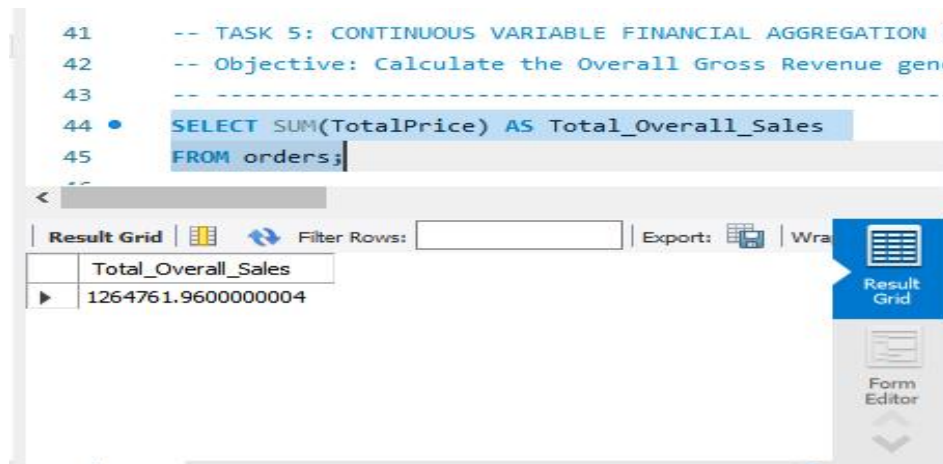
```
35  •  SELECT OrderID, CustomerID, Product, TotalPrice, Order
36      FROM orders
37      WHERE OrderStatus = 'Delivered' AND PaymentMethod = 'C
38
39
```

	OrderID	CustomerID	Product	TotalPrice	OrderStatus	Pa
▶	ORD200029	C33260	Desk	1270.59	Delivered	Cre
	ORD200033	C96664	Printer	769.38	Delivered	Cre
	ORD200065	C97735	Monitor	408.5	Delivered	Cre
	ORD200137	C50682	Monitor	501.96	Delivered	Cre
	ORD200145	C85929	Printer	440.73	Delivered	Cre
	ORD200171	C20448	Tablet	296.44	Delivered	Cre
	ORD200179	C27401	Laptop	1619.73	Delivered	Cre

Task 5: Continuous Variable Financial Aggregation (Total Sales)

SQL Query Used: `SELECT SUM(TotalPrice) AS Total_Overall_Sales FROM orders;`

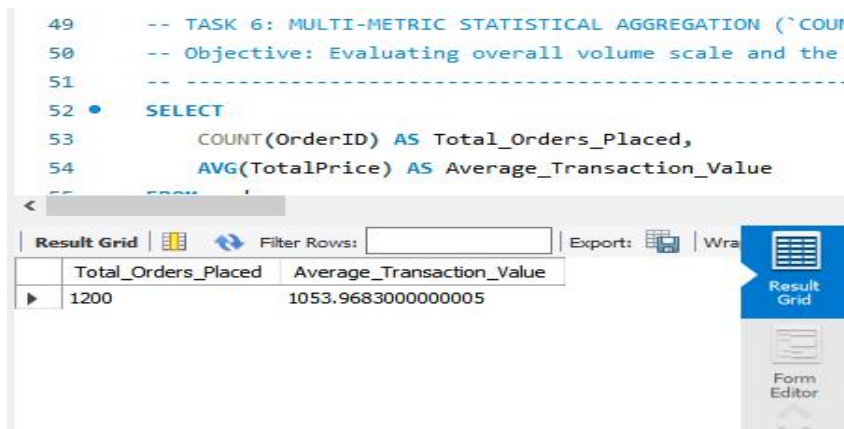
Business Insight: The company generated a total gross revenue of 1,264,761.96 across all processed 1,200 historical orders.



Task 6: Multi-Metric Statistical Aggregation

SQL Query Used: `SELECT COUNT(OrderID) AS Total_Orders_Placed, AVG(TotalPrice) AS Average_Transaction_Value FROM orders;`

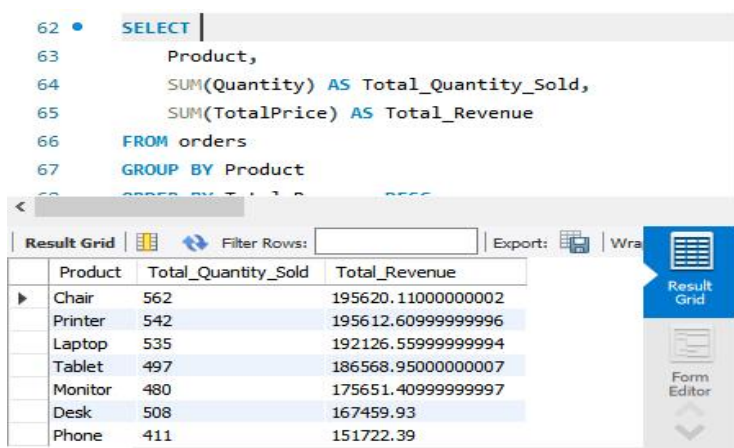
Business Insight: Automatically computes the enterprise operational scale. It validates a complete batch of 1,200 orders and determines the Average Transaction Value (ATV) to understand typical basket sizes.



Task 7: Dimensional Grouping & Product Performance Analysis

SQL Query Used: SELECT Product, SUM(Quantity) AS Total_Quantity_Sold, SUM(TotalPrice) AS Total_Revenue FROM orders GROUP BY Product ORDER BY Total_Revenue DESC;

Business Insight: Groups raw sales logs by product category to rank inventory performance. It reveals which tech items (e.g., Laptops, Monitors) serve as primary revenue engines versus lower-performing stock segments.



The screenshot shows a SQL query editor with the following query:

```
62 • SELECT
63     Product,
64     SUM(Quantity) AS Total_Quantity_Sold,
65     SUM(TotalPrice) AS Total_Revenue
66 FROM orders
67 GROUP BY Product
68 ORDER BY Total_Revenue DESC;
```

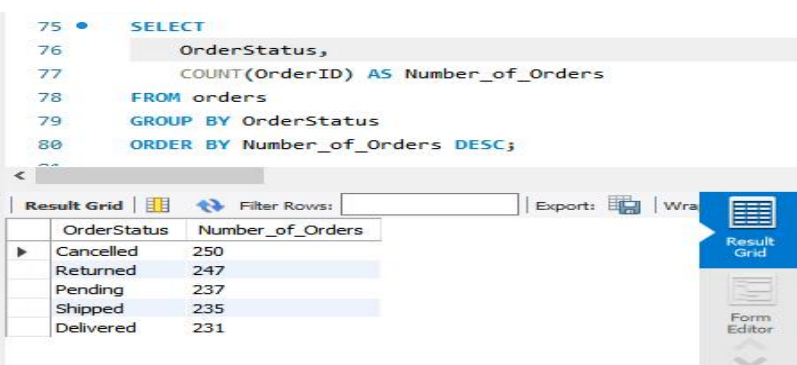
Below the query editor is a 'Result Grid' showing the results of the query. The grid has three columns: Product, Total_Quantity_Sold, and Total_Revenue. The results are sorted by Total_Revenue in descending order.

Product	Total_Quantity_Sold	Total_Revenue
Chair	562	195620.11000000002
Printer	542	195612.60999999996
Laptop	535	192126.55999999994
Tablet	497	186568.95000000007
Monitor	480	175651.40999999997
Desk	508	167459.93
Phone	411	151722.39

Task 8: Operational Logistics & Order Status Audit

SQL Query Used: SELECT OrderStatus, COUNT(OrderID) AS Number_of_Orders FROM orders GROUP BY OrderStatus ORDER BY Number_of_Orders DESC;

Business Insight: Provides the fulfillment team with operational metrics. By tracking the volume of 'Delivered', 'Pending', 'Cancelled', and 'Returned' orders, the business can identify bottlenecks in the supply chain pipeline.



The screenshot shows a SQL query editor with the following query:

```
75 • SELECT
76     OrderStatus,
77     COUNT(OrderID) AS Number_of_Orders
78 FROM orders
79 GROUP BY OrderStatus
80 ORDER BY Number_of_Orders DESC;
```

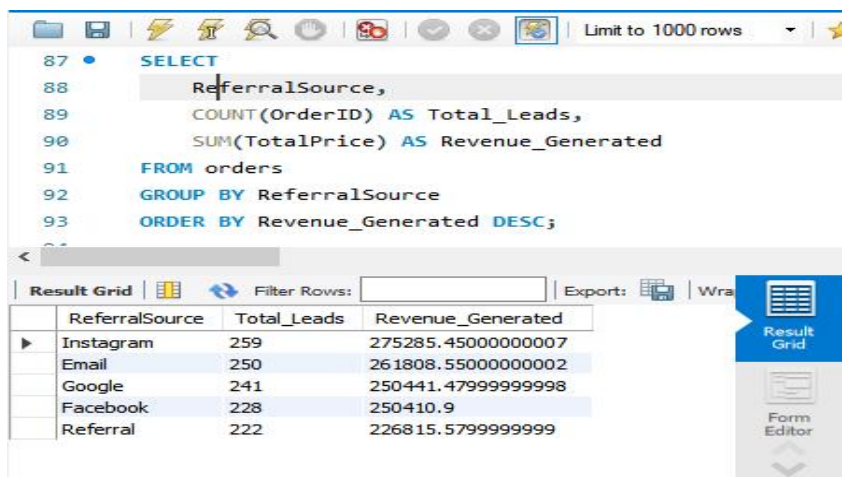
Below the query editor is a 'Result Grid' showing the results of the query. The grid has two columns: OrderStatus and Number_of_Orders. The results are sorted by Number_of_Orders in descending order.

OrderStatus	Number_of_Orders
Cancelled	250
Returned	247
Pending	237
Shipped	235
Delivered	231

Task 9: Marketing Acquisition Channel Conversion Metrics

SQL Query Used: SELECT ReferralSource, COUNT(OrderID) AS Total_Leads, SUM(TotalPrice) AS Revenue_Generated FROM orders GROUP BY ReferralSource ORDER BY Revenue_Generated DESC;

Business Insight: Maps corporate revenue drivers back to customer acquisition sources (Instagram, Google, Facebook, Email). This insight helps marketing teams optimize budget allocation toward the highest-converting digital platforms.



The screenshot shows a SQL query editor with the following query:

```
SELECT
  ReferralSource,
  COUNT(OrderID) AS Total_Leads,
  SUM(TotalPrice) AS Revenue_Generated
FROM orders
GROUP BY ReferralSource
ORDER BY Revenue_Generated DESC;
```

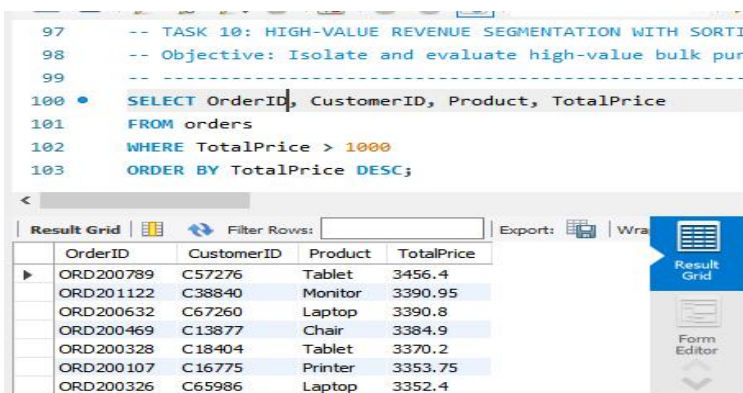
Below the query, a table grid displays the results:

ReferralSource	Total_Leads	Revenue_Generated
Instagram	259	275285.45000000007
Email	250	261808.55000000002
Google	241	250441.47999999998
Facebook	228	250410.9
Referral	222	226815.57999999999

Task 10: High-Value Revenue Segmentation with Sorting

SQL Query Used: SELECT OrderID, CustomerID, Product, TotalPrice FROM orders WHERE TotalPrice > 1000 ORDER BY TotalPrice DESC;

Business Insight: Isolates elite, high-value bulk purchases crossing the 1,000 threshold. Sorting these in descending order allows customer success teams to immediately flag and nurture VIP clients and enterprise transactions.



The screenshot shows a SQL query editor with the following query:

```
-- TASK 10: HIGH-VALUE REVENUE SEGMENTATION WITH SORTING
-- Objective: Isolate and evaluate high-value bulk purchases
SELECT OrderID, CustomerID, Product, TotalPrice
FROM orders
WHERE TotalPrice > 1000
ORDER BY TotalPrice DESC;
```

Below the query, a table grid displays the results:

OrderID	CustomerID	Product	TotalPrice
ORD200789	C57276	Tablet	3456.4
ORD201122	C38840	Monitor	3390.95
ORD200632	C67260	Laptop	3390.8
ORD200469	C13877	Chair	3384.9
ORD200328	C18404	Tablet	3370.2
ORD200107	C16775	Printer	3353.75
ORD200326	C65986	Laptop	3352.4